



Loyola Marymount
University

CMSI 5350/EECE 5150 Machine Learning

Professor Lanyu Shang

Announcement

- Homework 2 is available
 - Due next Friday 10/2

This lecture

- Machine learning fundamentals
 - Overfitting
 - Training/validation/testing
 - Hyperparameter tuning
 - Evaluation
- Decision Tree
 - Implementation with `scikit-learn`

Next

ML FUNDAMENTALS

ML as function approximation

- Problem setting
 - Set of possible **instances/samples** X
 - Set of possible **labels** Y
 - Unknown **target/ideal function** $f: X \rightarrow Y$
 - Set of **function hypotheses** $\mathcal{H} = \{h \mid h: X \rightarrow Y\}$
- Input: training data drawn i.i.d. from f
$$\mathcal{D}_n = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})\}$$
- Output: hypothesis $h \in \mathcal{H}$ that best approximates f

Decision tree – problem setting

- Set of possible instances $X = \{x^{(1)}, x^{(2)}, \dots, x^{(n)}\}$
 - Each instance $x^{(i)} \in X$ is a feature vector
 - E.g. $x^{(i)} = \langle \text{humidity}=\text{low}, \text{wind}=\text{weak}, \text{outlook}=\text{rain}, \text{temp}=\text{hot} \rangle$
- Set of possible labels $Y = \{y^{(1)}, y^{(2)}, \dots, y^{(n)}\}$
 - Y can be discrete, e.g., $y^{(i)} = \{'yes', 'no'\}$
 - Y can be continuous, e.g., $y^{(i)} \in \mathbb{R}$
- Unknown target function $f: X \rightarrow Y$
- Set of function hypotheses $\mathcal{H} = \{h \mid h: X \rightarrow Y\}$
 - Each hypothesis h is a decision tree
 - Tree sort $x^{(i)}$ to leaf, which assigns $y^{(i)} \in Y$

Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No

Learning a decision tree

- ML: heuristic search approach (ID3 algorithm)
 1. Start from the **root**
 2. Find the non-terminal node that results in the “**best**” **split**
 3. Create new **descendant of node**, sort/split the data
 4. **Repeat** step 2-3 until a stopping criterion is reached
- Information gain for the **best** splitting criterion
 - $I(Y, A) = H(Y) - H(Y|A)$
 - The difference between the **entropy before a split** and the **weighted average entropy after the split**.

Learning a decision tree

- Info gain: Y and each feature

- $I(Y, \text{Outlook}) = \mathbf{0.246}$ ✓

- $I(Y, \text{Temp}) = 0.029$

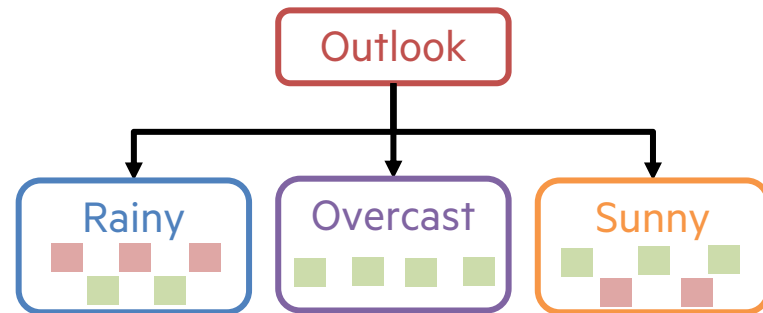
- $I(Y, \text{Humidity}) = 0.151$

- $I(Y, \text{Windy}) = 0.048$

- **Outlook** as the root node

- Next, determine the next split for each node in ➡

Predictors				Target
Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No



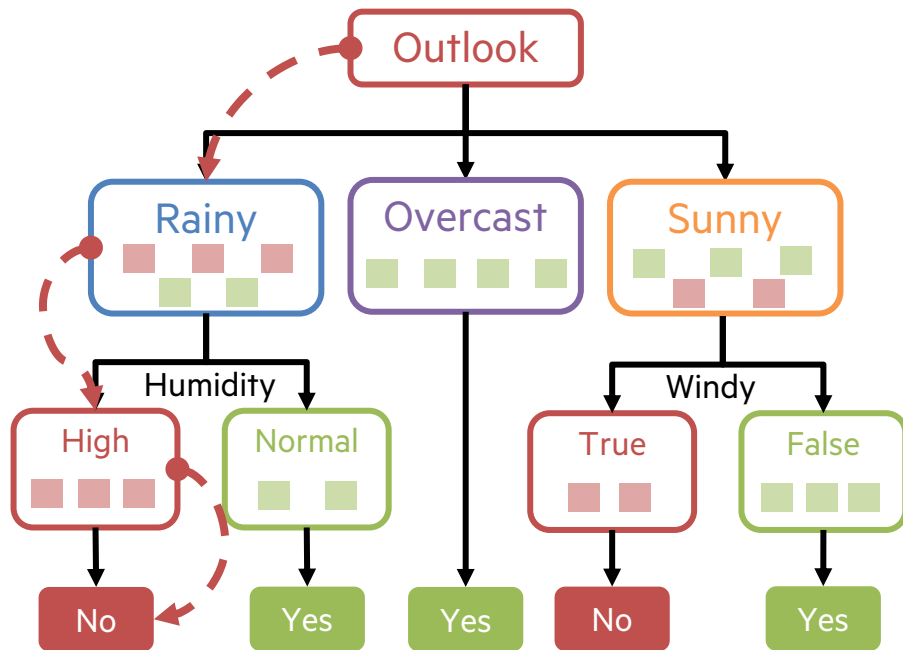
Learning a decision tree

Training data (X)

Predictors				Target
Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

Test data (X)

Outlook	Temp.	Humidity	Windy
Rainy	Hot	High	False



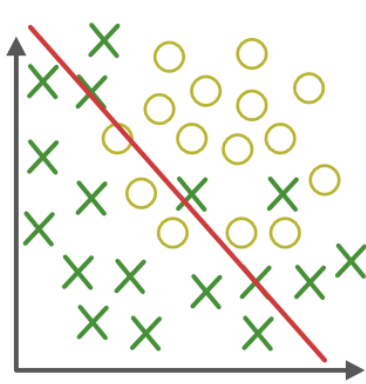
Prediction (y)

Overfitting

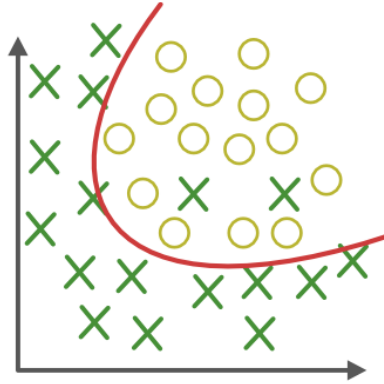
- Learning the training data **too perfectly**, including all its noise and random fluctuations, **rather than** learning the **true underlying patterns**.
 - Very **high accuracy** on training data
 - **Poor performance** on new, unseen data (validation/test sets)
- Examples
 - Memorizing exact answers to HW questions (training set) instead of understanding the concepts — failing on the exams (test set).

Overfitting vs. underfitting

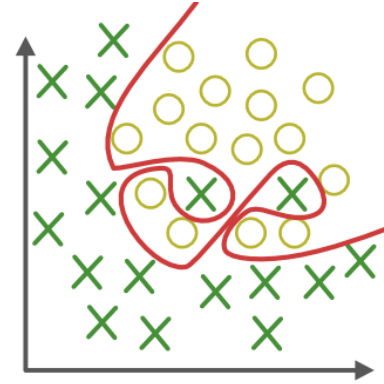
- Classification



Underfitting



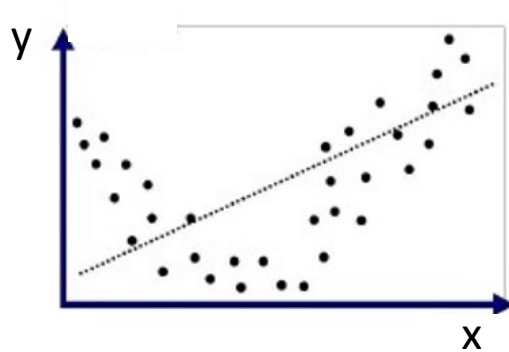
Good Fit



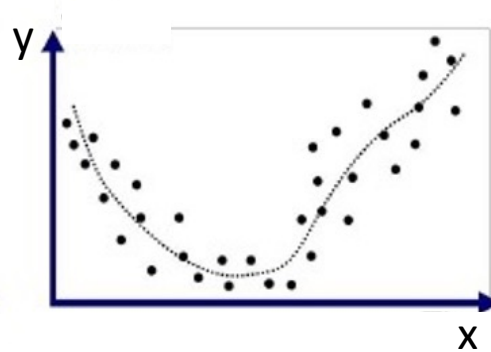
Overfitting

Overfitting vs. underfitting

- Regression



Underfitting

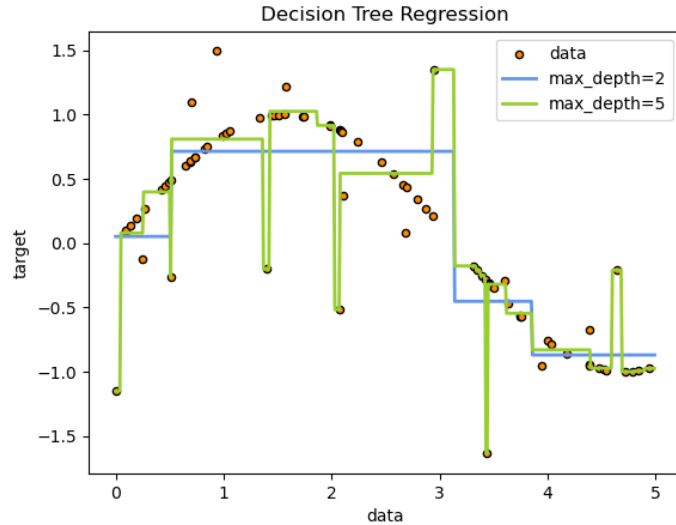


Good Fit

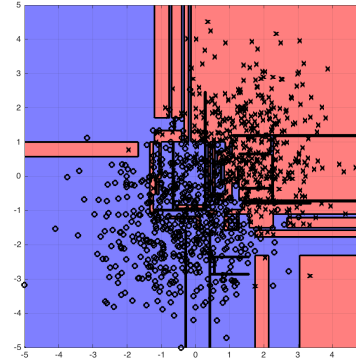


Overfitting

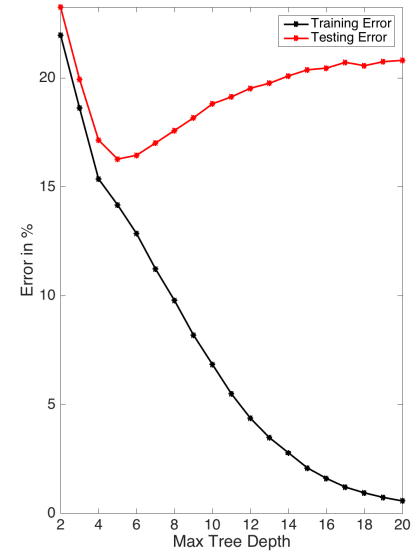
Overfitting in decision trees



Regression



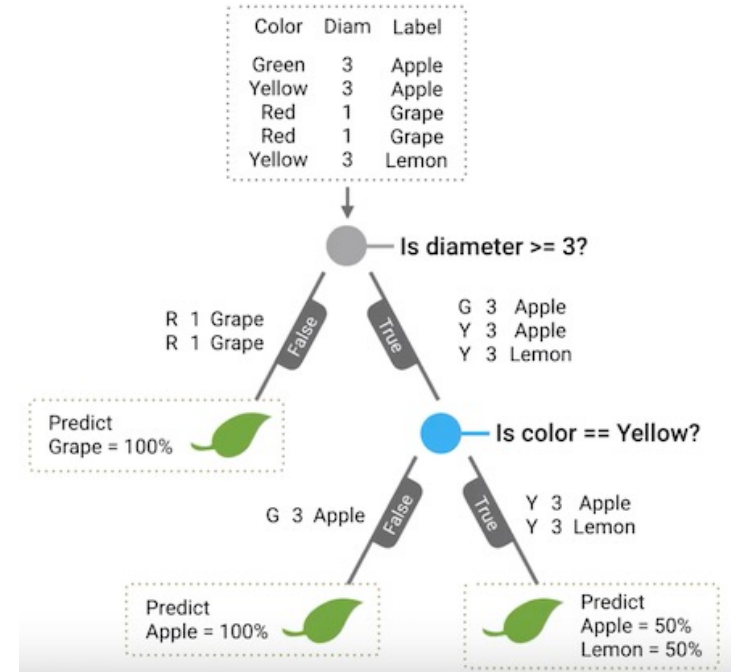
Classification



How to avoid overfitting in decision trees?

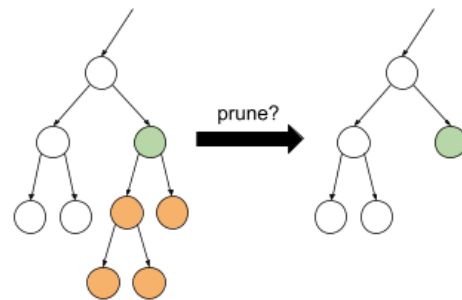
Overfitting in decision trees

- Pre-Pruning (early stopping)
 - Tree structure
 - Max tree depth
 - Min samples per split/leaf
 - Max leaf nodes
 - Splitting criteria
 - Min information gain decrease
 - Max features



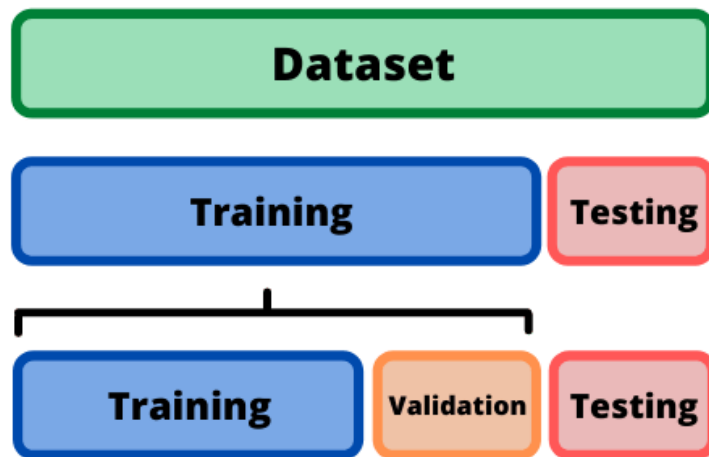
Overfitting in decision trees

- Post-Pruning (reducing nodes)
 - Removing “branches” that do not provide significant predictive power
- How?
 - For all internal nodes
 - remove subtree rooted at node
 - Assign class to be most common among training set
 - Check test data error
 - Remove the subtree if error is lower, otherwise restore it.

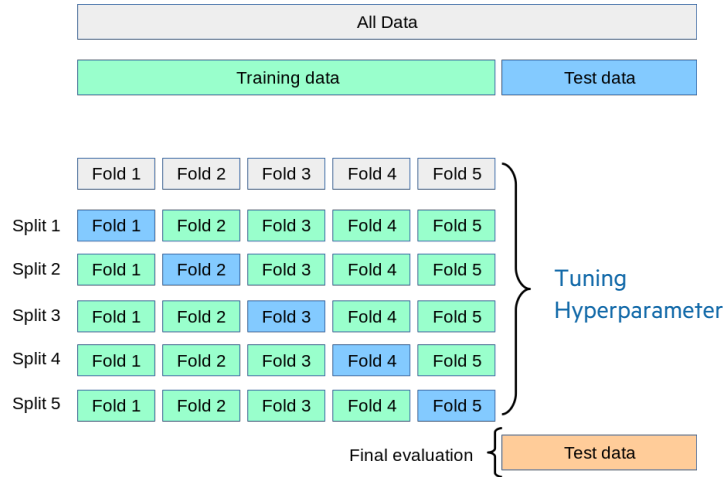


Training, validation, and testing

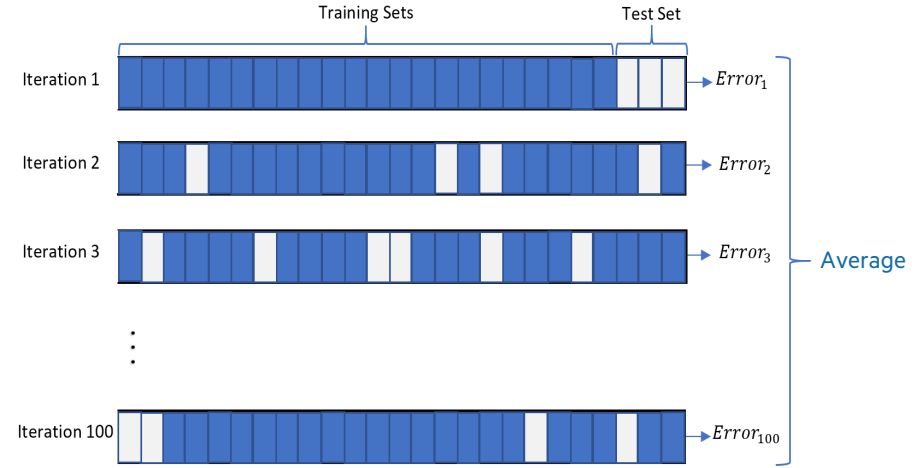
- Training (~70%)
 - Train/learn the model
- Validation (~20%)
 - Tune hyperparameters
 - Evaluate during training
- Testing (~10%)
 - For final model evaluation
 - Unbiased estimate of model performance



Cross-validation



K-Fold Cross-Validation



Monte Carlo Cross-Validation

What are the pros and cons?

Cross-validation

- Pros
 - More robust performance estimates than single train/test split
 - Optimize the utilization of training data
 - Detect overfitting and model stability issues
- Cons
 - Increased computational cost ($k * \text{training time}$)

Hyperparameter tuning

- Find the optimal values for a model's hyperparameters.
- **Parameters** and **hyperparameters** in decision trees
 - **Parameters**: automatically learned from the training data
 - Attribute selection at splits, split conditions
 - **Hyperparameters** : manually tuned via searching/optimizing
 - Maximum depth, min/max samples per split

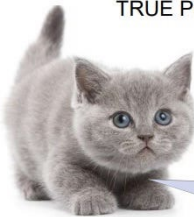
```
class sklearn.tree.DecisionTreeClassifier(*, criterion='gini',
splitter='best', max_depth=None, min_samples_split=2, min_samples_leaf=1,
min_weight_fraction_leaf=0.0, max_features=None, random_state=None,
max_leaf_nodes=None, min_impurity_decrease=0.0, class_weight=None,
ccp_alpha=0.0, monotonic_cst=None)
```

[\[source\]](#)

Evaluation – classification

- Confusion matrix

		Predicted	
		Spam	Non-spam
Actual	Spam	600 (True positive)	300 (False negative)
	Non-spam	100 (False positive)	9000 (True negative)

		PREDICTIVE VALUES	
		POSITIVE (CAT)	NEGATIVE (DOG)
ACTUAL VALUES	POSITIVE (CAT)	TRUE POSITIVE  3 YOU ARE A CAT	FALSE NEGATIVE  1 YOU ARE A DOG TYPE II ERROR
	NEGATIVE (DOG)	FALSE POSITIVE  2 YOU ARE A CAT TYPE I ERROR	TRUE NEGATIVE  4 YOU ARE NOT A CAT

Evaluation – classification

- Accuracy

- % of correct predictions

- $$\frac{TP+TN}{TP + TN + FP + FN}$$

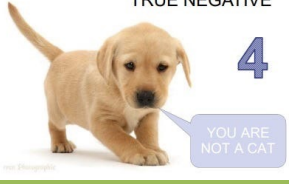
- Limitation

- Treats all errors equally

- Medical diagnosis: FN (missing a disease) could be life-threatening

- Class imbalance problem

- In a 99:1 dataset, 99% acc. may still miss all neg. samples

		PREDICTIVE VALUES	
		POSITIVE (CAT)	NEGATIVE (DOG)
ACTUAL VALUES	POSITIVE (CAT)	TRUE POSITIVE  3	FALSE NEGATIVE  1 TYPE II ERROR
	NEGATIVE (DOG)	FALSE POSITIVE  2 TYPE I ERROR	TRUE NEGATIVE  4

Evaluation – classification

- Precision
 - % of true positives among positive predictions
 - $\frac{TP}{TP + FP}$
 - Informative when **false positives** are costly
 - E.g., important email wrongly marked as spam

		PREDICTIVE VALUES	
		POSITIVE (CAT)	NEGATIVE (DOG)
ACTUAL VALUES	POSITIVE (CAT)	TRUE POSITIVE  3	FALSE NEGATIVE  1 TYPE II ERROR
	NEGATIVE (DOG)	FALSE POSITIVE  2 TYPE I ERROR	TRUE NEGATIVE  4

Evaluation – classification

- Recall
 - Sensitivity or true positive rate (TPR)
 - % of actual positives correctly identified
 - $$\frac{TP}{TP + FN}$$
 - Informative when **false negative** are costly
 - E.g., disease wrongly diagnosed as non-disease

		PREDICTIVE VALUES	
		POSITIVE (CAT)	NEGATIVE (DOG)
ACTUAL VALUES	POSITIVE (CAT)	TRUE POSITIVE 3  YOU ARE A CAT	FALSE NEGATIVE 1  YOU ARE A DOG TYPE II ERROR
	NEGATIVE (DOG)	FALSE POSITIVE 2  YOU ARE A CAT TYPE I ERROR	TRUE NEGATIVE 4  YOU ARE NOT A CAT

Evaluation – classification

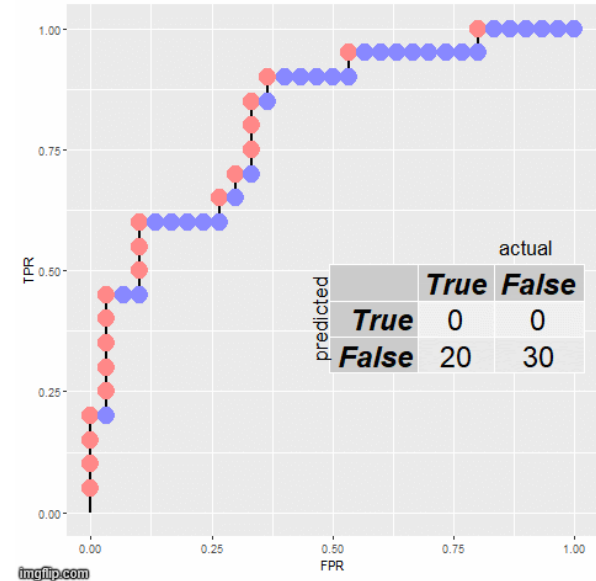
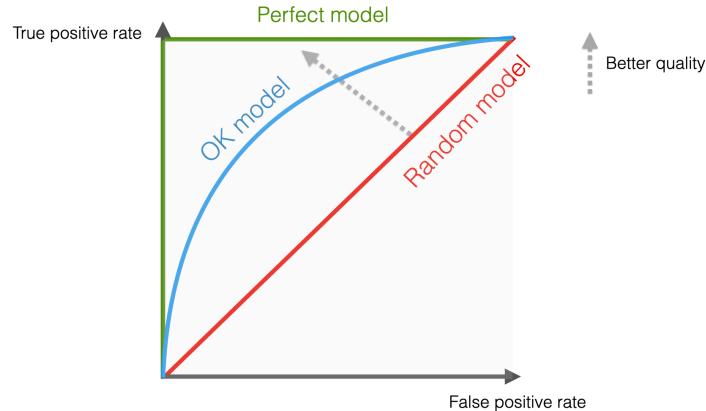
- F1 Score
 - **Harmonic** mean of precision and recall
 - $$F1\ Score = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}} = \frac{2 * (Precision * Recall)}{Precision + Recall}$$
 - Informative when the dataset is **imbalanced**
 - E.g., credit card fraud, rare disease

Evaluation – classification

- Receiver Operating Characteristic (ROC)

- ROC Curve:

- X-axis: $FPR = FP / (FP + TN)$
- Y-axis: $TPR = TP / (TP + FN)$



Evaluation – regression

- Mean Squared Error (MSE)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Root Mean Squared Error (RMSE)

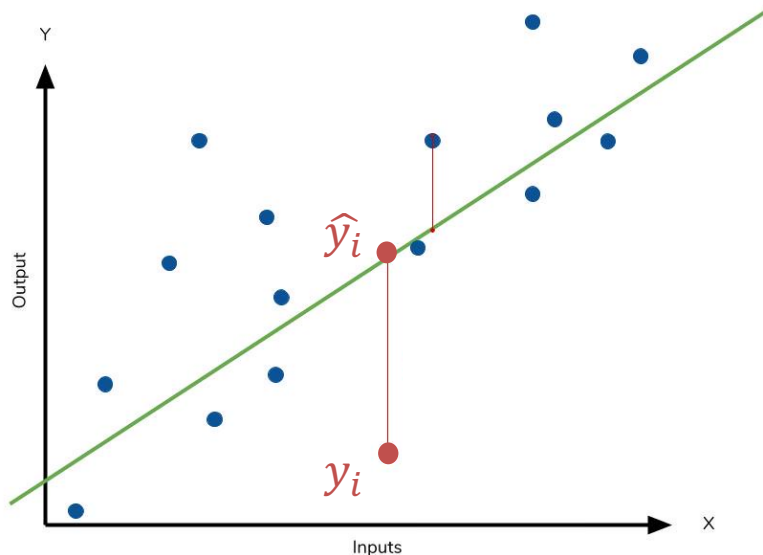
$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- Mean Absolute Error (MAE)

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- R-squared (R^2)

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$



Evaluation - other



Image segmentation

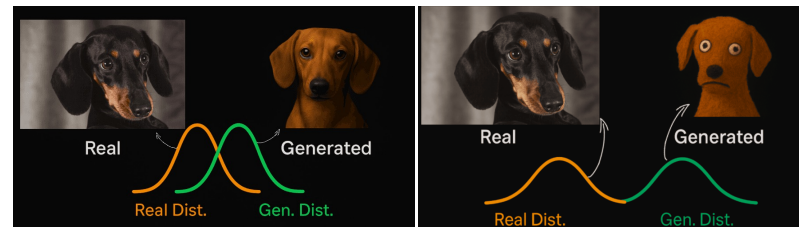
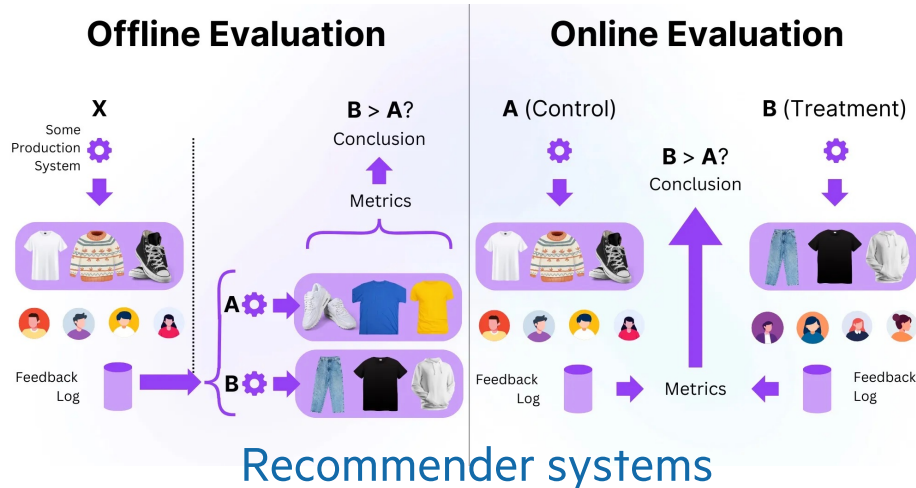
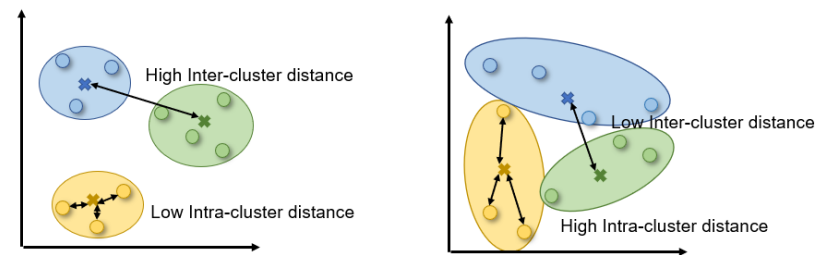


Image generation



Recommender systems



Clustering

Evaluation - other

- How about LLMs?

CATEGORY	EXAMPLE METRICS	PROS	CONS	BEST USE
Benchmarks	Accuracy, LogProb	Objective, standard	Can be outdated	General capability
Human-in-the-Loop	Elo, Ratings	Human insight	Costly, slow	Conversational or creative tasks
LLM-as-a-Judge	Rubric score	Scalable	Bias risk	Quick evaluation and A/B testing
Verifiers	Code/math checks	Objective	Narrow domain	Technical reasoning tasks
Reasoning-Based	PRM, ChainEval	Process insight	Complex setup	Agentic models
Text Quality	BLEU, ROUGE	Easy to automate	Overlooks semantics	NLG tasks
Safety/Bias	BBQ, SafeBench	Essential for ethics	Hard to quantify	Compliance and responsible AI

Colab notebook

DECISION TREE